Capstone E-Commerce Code Flow

Frontend and Backend Architecture, Service Interactions, Payment Webhook Flow, and Email/Kafka Flow

Workspace: `D:\Capstone`

Local frontend: `http://localhost:3000`

Local API Gateway: `http://localhost:8088`

---

1. Project Overview

This project is a microservices-based e-commerce system. The frontend is a static JavaScript application, while the backend is split into Spring Boot services. The frontend talks to the backend through API Gateway. The backend services register with Eureka Service Discovery, store data in MySQL, and use Kafka for asynchronous email messages.

Component	Local Port	Responsibility
Frontend	3000	User interface, routing, cart, checkout pages
API Gateway	8088	Single backend entry point for the frontend
Service Discovery	8761	Eureka registry for backend services
User Auth Service	8082	Signup, login, JWT/user workflows
Product Catalog Service	8084	Product listing, product detail, search
Order Service	8083	Order creation, lookup, status update
Payment Service	8085	Stripe checkout link and webhook handling
Email Service	8081	Consumes Kafka messages and sends email
Kafka	9092	Async messaging for signup and order emails
MySQL	3307 on host	Persistent storage for service databases

High-Level Architecture

User Browser → **Frontend localhost:3000** → **API Gateway localhost:8088**

API Gateway **routes to** Auth | Product | Order

Payment

Auth / Product / Order → MySQL **and** Kafka → Email Service

Payment Service ↔ Stripe **webhook via** **API Gateway**

2. Frontend Flow

The frontend app starts from `frontend/index.html`. It loads `js/app.js`, which handles hash-based routing. Routes such as `#/products`, `#/cart`, `#/checkout`, and `#/orders` render different pages.

```
index.html
  → js/app.js
  → route detection from URL hash
  → page renderer
  → API service modules
  → http://localhost:8088/* through API Gateway
```

Frontend File	Purpose
`frontend/js/config.js`	Stores API Gateway URLs for auth, products, orders, payments, and search.
`frontend/js/app.js`	Bootstraps the app and routes by hash.
`frontend/js/pages/products.js`	Loads and renders product cards.
`frontend/js/pages/cart.js`	Reads local cart data and displays cart contents.
`frontend/js/pages/checkout.js`	Creates an order and starts Stripe payment.
`frontend/js/api/*.js`	Wraps backend calls to API Gateway.

Frontend API URLs

```
AUTH_SERVICE:    http://localhost:8088/auth
PRODUCT_SERVICE: http://localhost:8088/products
ORDER_SERVICE:   http://localhost:8088/orders
PAYMENT_SERVICE: http://localhost:8088/api/payments
SEARCH_SERVICE:  http://localhost:8088/search
```

3. API Gateway Flow

API Gateway is the public backend entry point. The frontend does not need to know the internal service ports. Gateway forwards incoming requests to services discovered through Eureka.

Gateway Path	Target Service
`/auth/**`, `/users/**`	UserAuthService
`/products/**`, `/search/**`	ProductCatalogService
`/orders/**`	OrderService
`/api/payments/**`	PaymentService
`/api/stripewebhook`	PaymentService webhook controller
`/email/**`	EmailService

> Locally, API Gateway runs at `http://localhost:8088`. In cloud deployment, Stripe and the frontend must use the deployed public API Gateway domain.

4. Signup And Welcome Email Flow

> 1. User fills the signup form on the frontend.
> 2. Frontend sends `POST /auth/signup` to API Gateway.
> 3. Gateway routes the request to UserAuthService.
> 4. UserAuthService creates the user.
> 5. UserAuthService publishes an email event to Kafka topic `Signup`.
> 6. EmailService consumes the Kafka message.
> 7. EmailService sends a welcome email through Gmail SMTP.

```
User Browser
  → Frontend signup form
  → API Gateway /auth/signup
  → UserAuthService
  → Kafka topic: Signup
  → EmailService
  → Gmail SMTP
```

5. Product Browsing Flow

> 1. User clicks the Products page.
> 2. Frontend calls `GET http://localhost:8088/products`.
> 3. API Gateway forwards to ProductCatalogService.
> 4. ProductCatalogService reads products from MySQL database `product_catalog_service`.
> 5. Product DTOs are returned to the frontend.
> 6. Frontend renders product cards.

```
Products Page
  → ProductService.getAllProducts()
  → GET /products
  → ProductCatalogService
  → MySQL product table
  → JSON product list
  → Product cards
```

6. Cart Flow

The cart is handled on the frontend using browser local storage. Adding to cart does not immediately call the backend. The backend becomes involved during checkout, when the cart is converted into an order request.

```
User clicks Add to Cart
  → ProductCard component
  → StorageService.addToCart()
  → Browser localStorage
  → Navbar cart count updates
```

7. Checkout And Stripe Payment Flow

> 1. User submits the checkout form with name, email, phone, and address.
> 2. Frontend creates an order by calling `POST /orders`.
> 3. OrderService stores the order as `PENDING`.
> 4. Frontend calls `POST /api/payments`.
> 5. PaymentService creates a Stripe Checkout Session.
> 6. PaymentService returns the Stripe Checkout URL.
> 7. Frontend opens Stripe Checkout for the user.

```
Checkout form
  → POST /orders
  → OrderService saves order as PENDING
  → POST /api/payments
  → PaymentService creates Stripe session
  → Frontend opens Stripe payment page
```

8. Stripe Webhook And Order Confirmation Flow

This is the true payment confirmation path. Order creation and payment confirmation are intentionally separate. An order is created as `PENDING`. It becomes `CONFIRMED` only after Stripe sends a successful webhook event.

> 1. User completes payment on Stripe Checkout.
> 2. Stripe sends a webhook event to API Gateway at `/api/stripewebhook`.
> 3. Gateway routes the webhook to PaymentService.
> 4. PaymentService verifies the Stripe signature with `stripe.webhook.secret`.
> 5. PaymentService reads `orderId` from Stripe metadata.
> 6. PaymentService calls OrderService to update the order status to `CONFIRMED`.
> 7. OrderService updates MySQL and publishes a Kafka event.
> 8. EmailService consumes the event and sends the order confirmation email.

```
Stripe
  → POST /api/stripewebhook
  → PaymentService verifies event
  → PUT /orders/{id}/status?status=CONFIRMED
  → OrderService updates DB
  → Kafka topic: OrderCreated
  → EmailService sends confirmation email
```

> Local webhook URL: `http://localhost:8088/api/stripewebhook`. Stripe cannot call localhost directly unless you use Stripe CLI or a tunnel such as ngrok. In cloud, configure Stripe with `https://your-api-gateway-domain/api/stripewebhook`.

9. Email And Kafka Flow

Kafka decouples user-facing actions from email sending. Signup and order confirmation both use Kafka topics, but they are triggered at different moments.

Topic	Producer	Consumer	When It Fires
`Signup`	UserAuthService	EmailService	After successful user signup
`OrderCreated`	OrderService	EmailService	After order status changes to `CONFIRMED`

Latest Order Email Behavior

The order email flow was adjusted so the confirmation email is sent only after payment is confirmed, not when the order is merely created. The frontend passes `customerEmail` during checkout. OrderService stores that email and includes it in the Kafka event. EmailService uses that email as the recipient.

10. Service Responsibilities

Service	Main Responsibilities
API Gateway	Routes frontend and Stripe webhook traffic to backend services.
Service Discovery	Registers services and lets Gateway resolve service names.
UserAuthService	Handles signup/login and publishes signup email events.
ProductCatalogService	Manages product listing, detail, and search.
OrderService	Creates pending orders, updates statuses, publishes confirmed order email events.
PaymentService	Creates Stripe sessions and handles webhook events.
EmailService	Consumes Kafka email events and sends emails via Gmail SMTP.
Kafka	Message broker for async signup and order confirmation emails.
MySQL	Stores users, products, orders, and supporting service data.

11. End-To-End User Journey

```
1. User opens http://localhost:3000
2. User signs up
3. UserAuthService creates account
4. EmailService sends welcome email
5. User browses products
6. User adds products to cart
7. User checks out
8. OrderService creates a PENDING order
9. PaymentService creates Stripe payment link
10. User pays on Stripe
11. Stripe calls /api/stripewebhook
12. PaymentService marks order CONFIRMED through OrderService
13. OrderService publishes Kafka order event
14. EmailService sends order confirmation email
```

12. Cloud Deployment Notes

- Frontend should use the deployed API Gateway URL instead of `localhost:8088`.
- Stripe success and cancel URLs should use the deployed frontend domain instead of `localhost:3000`.
- Stripe webhook endpoint should be configured as `https://your-api-gateway-domain/api/stripewebhook`.
- The Stripe webhook signing secret must match the webhook endpoint configured in Stripe Dashboard.
- Secrets such as Gmail app password, Stripe API key, and webhook secret should be environment variables in cloud.

> In simple terms: the frontend is the storefront, API Gateway is the main backend entrance, Eureka is the service phonebook, MySQL stores business data, Kafka carries background email messages, EmailService sends mail, and Stripe confirms payment through the webhook.